

Esame OS Internals (Cabodi)

Data: 08/09/2022

QUESTIONS

SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale, Paginazione e Codice C)

Contesto: Sia dato il frammento di programma rappresentato:

```
#define N 128
#define M 32
int i, j, k;
float *A[N], B[N*M];

for (i = 0; i < N*M; i++) {
    B[i] = i;
}

for (i = 0; i < N; i++) {
    A[i] = calloc(i + 1, sizeof(float));
}

for (k = 0; k < N*M; k++) {
    i = rand() % N;
    j = rand() % i;
    A[i][j] += B[i + j*M];
}
```

Il codice macchina generato da tali istruzioni viene eseguito in un sistema con memoria virtuale gestita mediante paginazione, con pagine di 4 Kbyte, utilizzando come politica di sostituzione pagine la LRU (Least Recently Used). Si sa che i puntatori occupano 64 bit, mentre sia `int` che `float` hanno dimensione 32 bit e che le istruzioni in codice macchina, corrispondenti al frammento di programma visualizzato, sono contenute in una sola pagina. Si supponga che A e B siano allocati ad indirizzi logici contigui (prima A, poi B), a partire dall'indirizzo logico 0x74240000.

Si risponda alle seguenti domande:

-) Quante pagine (e frame) sono necessarie per contenere le strutture dati A e B, più la memoria allocata dinamicamente, supponendo (pur trattandosi di una semplificazione) che le chiamate alla `calloc` allochino in modo progressivo e contiguo nello heap, a partire dall'inizio di una pagina.
-) Ipotizzando che le variabili `i`, `j` siano allocate in registri della CPU, quanti accessi in memoria (in lettura e scrittura) fa il programma proposto, per accedere a dati (non vanno conteggiati gli accessi a istruzioni, né gli azzeramenti fatti dalla `calloc`, per la quale si suppone di poter ottenere pagine già azzerate; si trascurino inoltre le chiamate alla `rand()`)?

-) Calcolare il numero di page fault generati dal programma proposto, supponendo che siano allocati per esso 16 frame, di cui uno utilizzato (già all'inizio dell'esecuzione) per le istruzioni.
-

Domanda 2 (File System e Allocazione Indicizzata)

Contesto: Considerare un file system basato su allocazione indicizzata nel quale, per gestire i file di grandi dimensioni, si mettono in lista i blocchi indice. I puntatori/indici hanno una dimensione di 32 bit e i blocchi del disco hanno una dimensione di 4 KB. Il file system risiede su una partizione del disco di 600 GB, che include sia blocchi di dato che blocchi di indice.

Si risponda alle seguenti domande:

-) Dato un file binario di dimensione 10381 KB, calcolare esattamente quanti blocchi indice e blocchi di dato occupa il file. Calcolare anche la frammentazione interna, sia per i blocchi di dato che per quelli di indice.
 -) Si sa che il file contiene una sequenza di numeri reali in formato `double` (64 bit per numero). Si calcoli fino a quale blocco di indice e a quale riga (casella) di questo occorre accedere per leggere i primi 1M (un Mega) numeri del file. Si supponga che il puntatore al blocco indice successivo in lista sia posizionato all'inizio di un blocco indice. (Verificare che la dimensione sia consistente).
-

Domanda 3 (Politiche di Allocazione e Sostituzione)

Contesto: Si consideri un contesto di paginazione a richiesta con sostituzione di pagine. Si risponda alle domande seguenti.

- Spiegare brevemente la differenza tra "equal allocation" e "proportional allocation".
- Con quale delle due tecniche (ci si riferisce alla domanda A) si può realizzare una politica di sostituzione di tipo "working set"? (Motivare la risposta tra Equal allocation, Proportional allocation, Entrambe, Nessuna delle due).
- Considerando politiche di sostituzione di tipo "local" e "global":
 1. Quale delle due permette a un processo di non dipendere (o dipendere in modo minore) dagli altri processi in esecuzione (per quanto riguarda la sequenza di allocazioni/sostituzioni)? (Motivare).
 2. Perché una politica di tipo "global" può ottenere un miglior "throughput"?

SECTION 2: OS161

Domanda 4 (Sincronizzazione e spinlock)

Contesto: Si confrontino le due implementazioni (parziali) di `spinlock_acquire`, in OS161, fornite di seguito (`spinlock_acquire_v1` e `spinlock_acquire_v2`) e si risponda alle seguenti domande:

```
void spinlock_acquire_v1(struct spinlock *splk) {
    int again = 1;
    while (again) {
        if (spinlock_data_get(&splk->splk_lock) == 0) {
            if (spinlock_data_testandset(&splk->splk_lock) == 0) {
                again = 0;
            }
        }
    }
}

void spinlock_acquire_v2(struct spinlock *splk) {
    while (spinlock_data_testandset(&splk->splk_lock) != 0);
}
```

- Le due implementazioni sono (per la parte visibile) corrette, il che significa che implementano un comportamento di spinlock corretto? (In caso di errori, elencare gli errori; se sono corretti spiegarne il motivo).
 - Quali sono le principali differenze in termini di comportamento tra le due implementazioni? (Le diverse istruzioni sono chiaramente visibili, la domanda è cosa implementano operativamente).
 - Sulla base del codice visto e del funzionamento richiesto a uno spinlock, fornire una possibile definizione (linguaggio C) di `struct spinlock` (spiegare/motivare brevemente la propria definizione).
-

Domanda 5 (Gestione Processi, waitpid e PIDs)

Contesto: Si consideri, in OS161, la realizzazione della system call `waitpid`. Si risponda alle seguenti domande:

- Perché il processo che termina (con `sys_exit`) non può chiamare direttamente la `proc_destroy`, dopo aver segnalato la fine del processo stesso (con semaforo o condition variable) e prima della `thread_exit`?
- Si vogliono realizzare due funzioni (di cui si forniscono i prototipi) in grado di passare da puntatore a processo a pid e viceversa:

```
pid_t proc_to_pid(struct proc *p);
struct proc *proc_from_pid(pid_t pid);
```

Quesito: Si realizzino le due funzioni. NON è necessario realizzare inizializzazioni di strutture dati o altre funzioni. Ma è necessario spiegare a parole cosa siano (e come andrebbero inizializzate) eventuali strutture dati necessarie per effettuare le due conversioni.

ANSWERS

Domanda 1 (Memoria Virtuale, Paginazione e Codice C)

Dati Iniziali:

- $N = 128$; $M = 32$; $N \times M = 4096$.
- `int` e `float` = 4 Byte. Puntatori = 8 Byte.
- Pagina = 4 KB = 4096 Byte.
- Indirizzo di partenza A e B: 0x74240000 (Allineato alla pagina).

A) Pagine e Frame per A, B e Heap:

- **Strutture contigue A e B:**

L'array di puntatori A occupa $128 \times 8 \text{ B} = 1024 \text{ B}$.

L'array B occupa $128 \times 32 \times 4 \text{ B} = 16.384 \text{ B}$.

Essendo allocati in modo strettamente contiguo, la loro dimensione combinata in memoria è $1024 + 16.384 = \mathbf{17.408 \text{ Byte}}$.

Poiché partono dall'inizio di una pagina, il numero di pagine necessarie è $\lceil 17.408/4096 \rceil = \lceil 4,25 \rceil = \mathbf{5 \text{ pagine}}$. (Che in RAM corrisponderanno a **5 frame**).

- **Memoria allocata dinamicamente (Heap):**

Il ciclo `calloc` alloca $i + 1 \text{ float}$ per i che va da 0 a 127. Il numero totale di elementi allocati corrisponde alla somma dei primi 128 numeri naturali:

$$\text{Elementi} = \frac{N(N+1)}{2} = \frac{128 \times 129}{2} = 8.256 \text{ float.}$$

La dimensione totale è $8.256 \times 4 \text{ B} = \mathbf{33.024 \text{ Byte}}$.

Ipotizzando allocazione contigua a partire da una nuova pagina, le pagine richieste sono $\lceil 33.024/4096 \rceil = \lceil 8,06 \rceil = \mathbf{9 \text{ pagine (9 frame)}}$.

B) Accessi a memoria per i dati:

Assumendo che le variabili i e j si trovino nei registri della CPU (come da traccia) e trascurando gli accessi per la gestione della variabile k , calcoliamo gli accessi alle strutture dati principali:

1. **Primo ciclo for:** Esegue `B[i] = i`. Viene eseguita 1 scrittura per iterazione. $N \times M = 4096$ iterazioni $\implies \mathbf{4096 \text{ accessi}}$.
2. **Secondo ciclo for:** Esegue `A[i] = calloc(...)`. Viene eseguita 1 scrittura (del puntatore) per iterazione. $N = 128$ iterazioni $\implies \mathbf{128 \text{ accessi}}$.
3. **Terzo ciclo for:** Esegue `A[i][j] += B[i+j*M]`. Essendo A un array di puntatori, l'operazione richiede ben **4 accessi a dati** per iterazione:

- Lettura del valore $B[\dots]$
- Lettura del puntatore $A[i]$ dalla memoria
- Lettura del valore float in $A[i][j]$
- Scrittura del risultato in $A[i][j]$

$N \times M = 4096$ iterazioni $\times 4$ accessi $\Rightarrow$ **16.384 accessi**.

Totale accessi a dati: $4096 + 128 + 16.384 = 20.608$ accessi.

C) Calcolo del numero di Page Faults:

A disposizione abbiamo 16 frame, di cui 1 già occupato dalle istruzioni. Rimangono **15 frame liberi** per allocare i dati.

- Le strutture A e B occupano 5 pagine in totale.
- I dati dinamici nello Heap occupano 9 pagine.

L'intero processo necessita di **14 pagine di dati**. Poiché lo spazio richiesto per mappare l'intero *working set* (14 pagine) è strettamente inferiore ai frame fisicamente disponibili (15 frame), **la RAM è in grado di contenere l'intero set di dati senza mai innescare la politica di sostituzione LRU**.

Di conseguenza, si verificheranno esclusivamente i Page Fault "obbligatori" (*compulsory misses*) al primissimo accesso di ciascuna pagina allocata.

Totale Page Faults = Pagine A/B + Pagine Heap = $5 + 9 = 14$ **Page Faults**.

Domanda 2 (File System e Allocazione Indicizzata)

Dati Iniziali:

- Dimensione Blocco = 4 KB = 4096 Byte.
- Dimensione Puntatore = 32 bit = 4 Byte.
- Capacità lorda di un blocco indice = $4096/4 = 1024$ puntatori.
- **Capacità utile:** Trattandosi di una lista concatenata di blocchi indice, il primo puntatore (4 Byte) di ogni blocco indice è riservato per puntare al blocco indice successivo. Di conseguenza, i puntatori utili per mappare i dati sono **1023 per ogni blocco indice**.

A) Calcolo dei blocchi e della frammentazione interna:

1. Blocchi Dato:

- Dimensione file = 10381 KB.
- Numero blocchi dato = $\lceil \frac{10381 \text{ KB}}{4 \text{ KB}} \rceil = \lceil 2595,25 \rceil = 2596$ **blocchi**.
- Frammentazione interna = Spazio Allocato - Spazio Effettivo = $(2596 \times 4 \text{ KB}) - 10381 \text{ KB} = 10384 - 10381 = 3 \text{ KB}$ (**3072 Byte**).

2. Blocchi Indice:

- Essendoci 2596 blocchi dato da mappare, divisi per 1023 slot utili a blocco:
- Numero blocchi indice = $\lceil \frac{2596}{1023} \rceil = \lceil 2,537 \rceil = 3$ **blocchi indice**.

- Frammentazione interna indici = Il terzo blocco indice ospita 1 puntatore "next" e i restanti puntatori ai dati ($2596 - (2 \times 1023) = 2596 - 2046 = 550$ puntatori usati). Gli slot totali occupati nell'ultimo blocco sono $1 + 550 = 551$. Gli slot vuoti sono $1024 - 551 = 473$.
- Frammentazione = $473 \text{ slot vuoti} \times 4 \text{ Byte} = \mathbf{1892 \text{ Byte}}$.

B) Accesso in lettura per i primi 1M numeri double:

Un numero double a 64 bit occupa 8 Byte.

1. Calcolo del volume di dati:

- Dati da leggere = $1\text{M} \times 8 \text{ Byte} = 2^{20} \times 8 \text{ B} = \mathbf{8 \text{ MB (8192 KB)}}$.
- Numero di blocchi dato attraversati = $\frac{8192 \text{ KB}}{4 \text{ KB}} = \mathbf{2048 \text{ blocchi}}$.

2. Individuazione della casella nel blocco indice: Dobbiamo recuperare l'indirizzo dell'ultimo blocco di questa sequenza, ovvero il 2048-esimo blocco dato logico (che corrisponde all'indice logico del file, da 1 a 2048).

- Il **Primo Blocco Indice** mappa i primi 1023 blocchi dato (dall'1 al 1023).
- Il **Secondo Blocco Indice** mappa i successivi 1023 blocchi dato (dal 1024 al 2046).
- Il **Terzo Blocco Indice** mappa i blocchi dal 2047 in poi. Il blocco dato 2048 risiede in questo terzo blocco indice.

Sapendo che la casella di indice 0 (la prima riga) contiene il puntatore al blocco indice successivo:

- Riga 0: puntatore "next".
- Riga 1: puntatore al blocco dato 2047.
- **Riga 2 (la terza casella):** puntatore al blocco dato **2048**.

Conclusione: Occorre accedere al **Terzo Blocco Indice**, alla **casella di indice 2 (la terza riga)**.

Domanda 3 (Politiche di Allocazione e Sostituzione)

A) Differenza tra "equal allocation" e "proportional allocation":

Entrambe sono strategie di allocazione **statica** dei frame di memoria.

- **Equal allocation (Allocazione equa):** Assegna esattamente lo stesso numero di frame a tutti i processi in esecuzione, dividendo i frame liberi totali per il numero di processi, indipendentemente dalle loro effettive necessità o dimensioni.
- **Proportional allocation (Allocazione proporzionale):** Assegna a ciascun processo una quota di frame proporzionale alla sua dimensione virtuale (o alle sue necessità dichiarate). I processi più grandi ricevono più frame rispetto a quelli più piccoli.

B) Quale tecnica per una politica "Working Set"?

Risposta: Nessuna delle due.

Motivazione: Sia l'Equal che la Proportional Allocation assegnano un numero *fisso e invariabile* di frame a un processo per tutta la sua durata di esecuzione. Il modello del **Working Set**, invece, si basa sull'osservazione delle pagine referenziate nell'ultima finestra temporale Δ . Poiché il *Working Set Size* (WSS) di un processo varia dinamicamente durante l'esecuzione (in base ai cambi di località), esso richiede obbligatoriamente un'allocazione **dinamica** dei frame. Il sistema operativo deve poter assegnare nuovi frame quando il WSS cresce e rimuoverne quando il WSS si restringe.

C) Politiche "Local" e "Global":

1. 1. Indipendenza dei processi:

La politica che garantisce l'indipendenza è la **Local replacement**.

Motivazione: In una politica locale, quando un processo causa un Page Fault, il sistema operativo seleziona la pagina vittima *esclusivamente* tra i frame già assegnati a quel medesimo processo. Di conseguenza, il tasso di Page Fault di un processo dipende unicamente dal suo comportamento e dalla sua quota di memoria, rendendolo completamente immune alle fluttuazioni e alle allocazioni degli altri processi concorrenti.

2. 2. Miglior Throughput della politica Global:

Motivazione: Nelle politiche locali o statiche, un processo con un Working Set molto ampio potrebbe andare in *thrashing* (subire continui page fault) avendo esaurito i propri frame, mentre contemporaneamente un altro processo più piccolo o inattivo potrebbe avere frame assegnati ma inutilizzati. Una politica **Global** permette a un processo che necessita di molta memoria di "rubare" frame ai processi che non li stanno utilizzando intensamente. Questo bilanciamento dinamico a livello di intero sistema riduce il numero totale di Page Fault globali e i tempi di attesa per l'I/O sul disco, massimizzando l'utilizzo utile della CPU e quindi il *throughput* complessivo.

Domanda 4 (Sincronizzazione e spinlock)

A) Correttezza delle implementazioni:

SI, entrambe le implementazioni sono assolutamente corrette e garantiscono il comportamento atteso da uno spinlock (mutua esclusione).

Motivazione: Entrambe utilizzano l'istruzione hardware atomica **testandset** per acquisire il lock in modo sicuro. Il fatto che la V2 generi un'attesa attiva (*busy waiting*) saturando la CPU non è un errore, ma è la definizione e lo scopo stesso dello spinlock, che è progettato appositamente per brevi attese su sistemi multi-processore senza causare context switch.

B) Differenze operative tra le due implementazioni:

Le due implementazioni corrispondono a due pattern classici noti come **TAS** (Test-and-Set, V2) e **TTAS** (Test-and-Test-and-Set, V1).

- **spinlock_acquire_v2 (TAS):** Esegue continuamente un'istruzione atomica di scrittura/lettura. Nei sistemi multi-core, ogni tentativo di scrittura sul bus della memoria genera un segnale di *cache invalidation* verso tutti gli altri processori. Molti thread in attesa su questo spinlock intaseranno il bus di memoria, degradando le prestazioni hardware globali.
- **spinlock_acquire_v1 (TTAS):** Risolve il problema del traffico sul bus introducendo un ciclo di lettura semplice (**spinlock_data_get**) prima di tentare l'acquisizione atomica. Una lettura semplice non genera invalidazione: il thread in attesa entra in *spin* leggendo il valore "1" direttamente dalla sua **cache locale** ad altissima velocità. Solo quando il proprietario del lock lo rilascia (scrivendo 0), la cache viene invalidata, la **get** legge 0, e il thread esce dal loop interno per eseguire l'effettiva **testandset**. Questo riduce drasticamente l'overhead sul bus di sistema.

C) Possibile definizione in C di struct spinlock:

Sulla base del codice e dei requisiti in ambiente kernel (OS161), una possibile definizione è la seguente:

```
struct spinlock {
    volatile spinlock_data_t splk_lock; /* Il dato atomico dello spinlock */
    struct cpu *splk_holder;             /* CPU che detiene attualmente il lock */
    const char *splk_name;               /* Nome per scopi di debugging */
};
```


Motivazione dei campi:

- **splk_lock:** È il "cuore" del lock (utilizzato nei frammenti forniti). Contiene lo stato (0 libero, 1 occupato). L'uso della keyword **volatile** è fondamentale per impedire al compilatore C di ottimizzare il ciclo **while**, forzandolo a rileggere sempre il valore reale dalla memoria.
- **splk_holder:** Memorizza un puntatore al processore (o thread) che ha acquisito il lock con successo. È fondamentale per la robustezza del sistema: permette al kernel di invocare **panic()** se un thread tenta di rilasciare uno spinlock che non possiede, oppure se tenta di riacquisire uno spinlock che già possiede (causando un deadlock).
- **splk_name:** Stringa testuale usata tipicamente in OS161 per facilitare la diagnostica e i log di sistema in caso di stallo o errori.

Domanda 5 (Gestione Processi, waitpid e PIDs)

A) Perché non chiamare **proc_destroy** in **sys_exit**?

Il processo non può distruggere se stesso prima di aver terminato definitivamente la propria esecuzione per due motivi fondamentali:

1. **Use-After-Free (Sicurezza):** Il thread che invoca **sys_exit** sta eseguendo codice in modalità kernel e necessita del proprio stack del kernel e delle strutture del processo (tra cui l'indirizzamento e i thread associati) per completare l'operazione di uscita e cedere la CPU (*context switch*). Se chiamasse **proc_destroy**, deallocherebbe la memoria che sta attualmente utilizzando, causando un *Kernel Panic*. Le strutture finali possono essere deallocate solo dopo l'ultimo context switch.
2. **Stato Zombie e waitpid:** Il processo deve entrare in uno stato "Zombie". L'architettura prevede che la struttura **proc** sopravviva alla morte del thread per conservare l'*exit status* (valore di terminazione). Sarà il processo padre, invocando la **waitpid()**, a leggere tale valore e a chiamare la vera **proc_destroy** sul figlio, ripulendo definitivamente la memoria.

B) Realizzazione funzioni di conversione PID ↔ Processo:

Strutture dati necessarie (spiegazione): Per supportare la conversione, occorre definire a livello globale nel kernel una **Process Table**.

- Un array globale di puntatori: `struct proc *ptable[PID_MAX];`. L'indice dell'array corrisponde direttamente al PID del processo.
- Un **Lock** di sincronizzazione: `struct lock *ptable_lock;` per proteggere l'array da accessi e modifiche concorrenti.
- *Inizializzazione:* Al boot del sistema, l'array deve essere riempito con valori **NULL** e il lock deve essere istanziato.

Realizzazione del codice C:

```
/* Converta un puntatore a processo nel suo PID */
pid_t proc_to_pid(struct proc *p) {
    if (p == NULL) {
        return -1; /* Errore: puntatore nullo */
    }
}
```

```

lock_acquire(ptable_lock);

/* Scansione lineare dell'array alla ricerca del puntatore */
/* Assumiamo che il PID 0 sia riservato/non valido */
for (int i = 1; i < PID_MAX; i++) {
    if (ptable[i] == p) {
        lock_release(ptable_lock);
        return (pid_t)i; /* PID trovato */
    }
}

lock_release(ptable_lock);
return -1; /* Errore: Processo non trovato nella tabella */
}

/* Recupera il puntatore al processo a partire dal PID */
struct proc *proc_from_pid(pid_t pid) {
    struct proc *p = NULL;

    /* Controllo di validità dei limiti del PID */
    if (pid <= 0 || pid >= PID_MAX) {
        return NULL; /* Errore: PID out of bounds */
    }

    lock_acquire(ptable_lock);

    /* Accesso diretto (O(1)) alla Process Table */
    p = ptable[pid];

    lock_release(ptable_lock);

    return p; /* Ritorna il puntatore (o NULL se l'entry è vuota) */
}

```